

RECEP TAYYIP ERDOGAN UNIVERSITY

Faculty of Engineering and Architecture

Department of Computer Engineering

EMBEDDED SYSTEMS

AMBULANCE SIMULATION SYSTEM

ESP32-S3 Based Multi-Mode Ambulance

Light and Siren Simulator

Student Name: Yakup Eroglu

Student Number: 221401045

Course: Embedded Systems

Platform: ESP32-S3 | PlatformIO

Date: June 2026

TABLE OF CONTENTS

1 Project Overview

2 Hardware Components

2.1 ESP32-S3-DevKitC-1

2.2 Joystick Module

2.3 Speaker Module

2.4 NeoPixel LED Ring

3 Circuit Connections

4 Software Architecture

5 Operating Modes

5.1 Mode 0 -- Off

5.2 Mode 1 -- Classic

5.3 Mode 2 -- Full Color

5.4 Mode 3 -- Slow Flash

6 Function Reference

7 Timing & Non-Blocking Design

8 PWM Audio Generation

9 NeoPixel LED Control

10 Button Debounce

11 Development Environment

12 Serial Monitor Output

13 Known Limitations

14 Future Development

1. PROJECT OVERVIEW

This project is a multi-mode ambulance light and siren simulation system developed using the ESP32-S3 microcontroller on the PlatformIO platform with the Arduino framework. The system cycles through four distinct operating modes controlled by a joystick push button, providing realistic ambulance-style LED animations and siren sounds.

Objectives

- Design and implement a real-time embedded system using the ESP32-S3 microcontroller
- Demonstrate non-blocking, millis()-based software architecture without delay() in the main loop
- Control addressable WS2812B RGB LEDs with multiple independent animation patterns
- Generate PWM-based audio tones simulating ambulance siren sweeps
- Implement reliable button input with 50ms software debounce algorithm
- Structure a modular, readable codebase following embedded C++ best practices

System Flow

```
Power On
|
v
setup() -> Initialize Hardware -> Startup Animation -> MODE 0 (Off)
|
v
loop() [runs continuously]
|-- Button pressed? --> enterMode(nextMode) [0->1->2->3->0...]
|-- MODE 1 --> updateMode1(now) [split LED flash + wee-woo siren]
|-- MODE 2 --> updateMode2(now) [full color LED + double beep]
+-- MODE 3 --> updateMode3(now) [3s blocks + heavy siren]
```

2. HARDWARE COMPONENTS

2.1 ESP32-S3-DevKitC-1

The ESP32-S3-DevKitC-1 is the central processing unit, built around the Espressif ESP32-S3 SoC. It features a dual-core Xtensa LX7 processor at 240 MHz and includes a built-in WS2812B RGB LED on GPIO48, enabling visual feedback without additional hardware.

Specification	Value
Processor	Xtensa LX7 dual-core, 240 MHz
RAM	320 KB SRAM
Flash	8 MB (Quad SPI)
Built-in RGB	GPIO48 (WS2812B)
ADC	12-bit, 20 channels
PWM (LEDC)	8 channels, up to 14-bit
USB	USB-OTG

2.2 Deneyap Joystick Module

Two-axis analog joystick with integrated push button. Only the SW push button is used in this project. X and Y axes are

wired but reserved for future enhancements.

Pin	GPIO	Function
SW (Button)	GPIO3	Mode toggle -- INPUT_PULLUP
X (Analog)	GPIO1	Not used (reserved)
Y (Analog)	GPIO2	Not used (reserved)
3V3	3.3V	Power supply
GND	GND	Common ground

2.3 Deneyap Speaker Module

Compact speaker + amplifier module. Driven by PWM from LEDC. The SD (shutdown) pin is actively driven LOW in MODE 0 to eliminate idle noise and save power.

Pin	GPIO	Function
SD	GPIO4	OUTPUT -- HIGH=active, LOW=shutdown
IN+	GPIO5	PWM Channel 0 audio signal
GND	GND	Common ground

2.4 WS2812B NeoPixel LED Ring (8 LEDs)

8 individually addressable RGB LEDs in a circular layout. 24-bit color per LED. VCC must be 5V -- 3.3V causes color distortion and insufficient brightness.

Specification	Value
LED Count	8
Protocol	WS2812B single-wire, 800 KHz
Color Depth	24-bit RGB (8-bit per channel)
VCC	5V (signal 3.3V tolerant)
Max Current	~480 mA (all white)
Library	Adafruit NeoPixel 1.15.5

3. CIRCUIT CONNECTIONS

Module	Module Pin	ESP32-S3 GPIO	Mode	Description
Joystick	3V3	3V3	Power	Supply voltage
Joystick	GND	GND	Ground	Common ground
Joystick	SW	GPIO3	INPUT_PULLUP	Push button (mode toggle)
Joystick	X	GPIO1	ADC (unused)	X-axis analog
Joystick	Y	GPIO2	ADC (unused)	Y-axis analog
Speaker	GND	GND	Ground	Common ground
Speaker	SD	GPIO4	OUTPUT	Enable/shutdown control
Speaker	IN+	GPIO5	PWM CH0	Audio square wave signal
LED Ring	GND	GND	Ground	Common ground
LED Ring	VCC	5V	Power	5V from USB
LED Ring	DIN	GPIO6	NeoPixel	WS2812B data signal
Built-in	--	GPIO48	Output	On-board RGB LED

ESP32-S3-DevKitC-1

```

+ -----+
| 3V3 ----- Joystick 3V3 |
| GND ----- Joystick GND |
| GPIO3 [INPUT_PULLUP]-- Joystick SW |
| GPIO4 [OUTPUT] ----- Speaker SD |
| GPIO5 [PWM CH0] ----- Speaker IN+ |
| GPIO6 [NeoPixel] ----- LED Ring DIN |
| 5V ----- LED Ring VCC |
| GPIO48 [Built-in RGB] |
+ -----+

```

4. SOFTWARE ARCHITECTURE

Pin Definitions

```

#define JOY_SW_PIN      3    // Joystick push button
#define SPEAKER_SD_PIN  4    // Speaker enable/shutdown
#define SPEAKER_PWM_PIN 5    // Speaker PWM signal
#define LED_PIN         6    // WS2812B NeoPixel data
#define LED_COUNT       8    // Number of LEDs
#define RGB_LED_PIN     48    // Built-in RGB LED
#define PWM_CHANNEL     0    // LEDC channel
#define PWM_RESOLUTION  8    // 8-bit resolution

```

Key Global Variables

Variable	Type	Role
currentMode	int	Active mode number (0-3)
lastBtnState / debouncedBtn	bool	Debounce state tracking
lastDebounce	unsigned long	Debounce timer (ms)

lastLedUpdate	unsigned long	LED animation timer (ms)
flashState	bool	LED color phase flag
lastSoundUpdate	unsigned long	Sound update timer (ms)
currentFreq / sirenDir	int	Siren sweep state
bipPhase / bipTimer	int / ulong	Mode 2 double-beep state machine
blockStart / blockRed	ulong / bool	Mode 3 3-second block state

5. OPERATING MODES

The system cycles through four modes with each button press: MODE 0 -> 1 -> 2 -> 3 -> 0 ...

Mode 0 -- Off (Standby)

Default state on power-on. All outputs are completely inactive. Speaker SD is LOW (amplifier shutdown). Only the button is polled each loop iteration.

Mode 1 -- Classic Ambulance

Simulates a European ambulance. LEDs 0-3 and LEDs 4-7 alternate between blue and red every 120 ms. Siren sweeps from 700 Hz to 1200 Hz and back (~1.75 Hz full cycle).

Feature	Detail
LED Pattern	Left 4: Blue, Right 4: Red -- sides swap every 120 ms
Siren Range	700 Hz to 1200 Hz
Sweep Step	14 Hz per 8 ms update
Full Wee-Woo Cycle	~572 ms

Mode 2 -- Full Color Loop

All 8 LEDs flash together: full blue then full red every 200 ms. Audio plays a two-tone double-beep pattern on a 780 ms cycle.

Beep Phase	Frequency	Duration
Phase 0 (Tone 1)	900 Hz	150 ms
Phase 1 (Silence)	--	80 ms
Phase 2 (Tone 2)	600 Hz	150 ms
Phase 3 (Silence)	--	400 ms
Total Cycle	--	780 ms

Mode 3 -- Slow Flash

All LEDs hold solid red for 3 seconds, then switch to solid blue for 3 seconds. Siren sweeps slowly from 500 Hz to 1000 Hz, completing one cycle every ~3 seconds to synchronize with the LED color changes.

Feature	Detail
LED Pattern	All LEDs solid RED for 3s, then solid BLUE for 3s
Siren Range	500 Hz to 1000 Hz
Sweep Step	6 Hz per 18 ms update
Full Sweep Cycle	~3000 ms (synchronized with LED)

6. FUNCTION REFERENCE

`setRGB(r,g,b)`

Controls GPIO48 built-in RGB LED via `neopixelWrite()`. No library needed.

`speakerOn(freq)`

Sets LEDC CH0 frequency via `ledcWriteTone()`. Minimum 20 Hz guard included.

`speakerOff()`

Sets LEDC duty to 0. Does not change SD pin state.

`allLed(r,g,b)`

Fills all 8 NeoPixels and calls `ring.show()` to commit.

`ledOff()`

Clears ring and turns off built-in RGB. Called on MODE 0 entry.

`enterMode(mode)`

Resets all state variables, controls SD pin, activates new mode cleanly.

`updateModel1(now)`

MODE 1 logic: LED every 120ms, sweep every 8ms -- fully non-blocking.

`updateMode2(now)`

MODE 2 logic: LED every 200ms, 4-phase beep state machine.

`updateMode3(now)`

MODE 3 logic: LED block every 3000ms with static change detection.

`readButtonPressed()`

50ms debounce; returns true only once per physical button press.

`setup()`

One-time init: serial, GPIO modes, LEDC, NeoPixel, RGB, startup animation.

`loop()`

Polls button, dispatches to active mode update function, `delay(1)` yields CPU.

7. TIMING AND NON-BLOCKING DESIGN

The `delay()` function halts all program execution, making simultaneous LED, audio, and button management impossible. All timing uses `millis()`-based non-blocking checks:

```
// WRONG -- button and sound cannot run during this 120ms freeze:
ring.fill(blue); ring.show(); delay(120);

// CORRECT -- returns in <1ms, everything keeps running:
if (now - lastLedUpdate >= 120) {
    lastLedUpdate = now;
    // update LED
}
```

Task	Interval	Variable
Mode 1 LED flash	120 ms	<code>lastLedUpdate</code>
Mode 1 siren sweep	8 ms	<code>lastSoundUpdate</code>
Mode 2 LED flash	200 ms	<code>lastLedUpdate</code>

Mode 2 beep phase	80-400 ms	bipTimer
Mode 3 LED block	3000 ms	blockStart
Mode 3 siren sweep	18 ms	lastSoundUpdate
Button debounce	50 ms	lastDebounce

8. PWM AUDIO GENERATION

The ESP32-S3 LEDC peripheral generates square wave tones without any external DAC. The square wave harmonic content actually enhances the perceived harshness, making the siren more realistic. The legacy LEDC API is used because the installed platform version (espressif32 @ 7.0.1) predates the unified ledcAttachChannel() function.

```
// Setup (once in setup()):
ledcSetup(PWM_CHANNEL, 2000, PWM_RESOLUTION); // channel, initial freq, resolution
ledcAttachPin(SPEAKER_PWM_PIN, PWM_CHANNEL); // attach GPIO5

// Tone on/off:
ledcWriteTone(PWM_CHANNEL, 880); // 880 Hz square wave
ledcWrite(PWM_CHANNEL, 0); // silence (duty = 0)

// Sweep algorithm (runs every N ms in loop):
currentFreq += sirenDir * STEP;
if (currentFreq >= MAX) { currentFreq = MAX; sirenDir = -1; }
if (currentFreq <= MIN) { currentFreq = MIN; sirenDir = +1; }
speakerOn(currentFreq);
```

Mode	Min Hz	Max Hz	Step	Interval	Character
Mode 1	700	1200	14 Hz	8 ms	Fast wee-woo (~1.75 Hz)
Mode 3	500	1000	6 Hz	18 ms	Slow heavy siren (~0.33 Hz)

9. NEOPIXEL LED CONTROL

WS2812B LEDs use a single-wire 800 KHz serial protocol. Each LED receives 24 bits (R, G, B each 8 bits) and passes remaining data downstream. ring.show() must always be called after modifying LED values -- changes are buffered until then.

```
Adafruit_NeoPixel ring(8, LED_PIN, NEO_GRB + NEO_KHZ800);
ring.begin();
ring.setBrightness(130); // ~51% brightness
ring.setPixelColor(i, ring.Color(r,g,b)); // single LED
ring.fill(ring.Color(255, 0, 0)); // all LEDs
ring.clear(); // all off
ring.show(); // commit to hardware (REQUIRED)
```

10. BUTTON DEBOUNCE

Mechanical contacts produce rapid signal bouncing (1-50ms) on every press. A 50ms millis-based debounce filters noise and returns true only once per press:

```
bool readButtonPressed() {
    bool reading = digitalRead(JOY_SW_PIN);
    if (reading != lastBtnState) { lastDebounce = millis(); lastBtnState = reading; }
    if ((millis() - lastDebounce) > DEBOUNCE_MS) {
        if (reading != debouncedBtn) {
            debouncedBtn = reading;
            if (debouncedBtn == LOW) return true; // ONE event per press
        }
    }
    return false;
}
```

11. DEVELOPMENT ENVIRONMENT

Software	Version	Purpose
Visual Studio Code	Any	Code editor
PlatformIO IDE	Any	Build and upload system
Espressif32 Platform	7.0.1	ESP32 toolchain
Arduino Framework	3.20017.241212	Arduino compatibility
Adafruit NeoPixel	1.15.5	WS2812B LED driver

```
[env:esp32-s3-devkitc-1]
platform = espressif32
board    = esp32-s3-devkitc-1
framework = arduino
lib_deps = adafruit/Adafruit NeoPixel @ ^1.12.3
monitor_speed = 115200
upload_speed = 921600
```

12. SERIAL MONITOR OUTPUT

```
Ambulance v2 -- 3 Modes + Off
  Button: MODE 0->1->2->3->0->...
  MODE 0: Off
  MODE 1: Classic ambulance
  MODE 2: Full blue-red + double beep
  MODE 3: 3s blocks + heavy siren
Ready! MODE 0 (Off). Press button.

MODE 1 active
MODE 2 active
MODE 3 active
  -> RED
  -> BLUE
MODE 0 -- Off
```

13. KNOWN LIMITATIONS

- Legacy LEDC API: upgrading espressif32 to 3.x requires switching to ledcAttachChannel().
- Square wave audio has harmonic distortion; I2S + sine LUT needed for clean audio.
- Joystick X/Y not read -- wired but reserved for future speed/brightness control.
- No persistent mode storage; system always starts in MODE 0 after power cycle.

14. FUTURE DEVELOPMENT

Short Term

- Joystick X/Y reading for siren speed and LED brightness control
- Mode 4: Rainbow HSV spectrum animation
- NVS persistent mode memory

Medium Term

- OLED display (SSD1306) for mode and frequency readout
- Bluetooth BLE mode control from smartphone
- I2S DAC (MAX98357A) for cleaner siren audio

Long Term

- MAX30102 heart rate integration
- Wi-Fi web interface (AsyncWebServer)
- Battery + deep-sleep for portable operation

QUICK REFERENCE CARD

Ambulance Simulation System -- ESP32-S3

PIN ASSIGNMENTS

BUTTON (SW)	GPIO3	INPUT_PULLUP -- Mode toggle
SPEAKER SD	GPIO4	OUTPUT -- Enable/shutdown
SPEAKER IN+	GPIO5	PWM Channel 0
LED DATA	GPIO6	NeoPixel DIN
BUILT-IN RGB	GPIO48	neopixelWrite()

OPERATING MODES

MODE 0	OFF	All outputs inactive -- standby
MODE 1	CLASSIC	Split blue/red flash + wee-woo siren
MODE 2	FULL COLOR	All blue <-> red + double beep
MODE 3	SLOW FLASH	3s color blocks + heavy slow siren

TURKCE DOKUMANTASYON

Ambulans Simulasyon Sistemi -- Proje Raporu

RECEP TAYYIP ERDOGAN UNIVERSITY

Muhendislik ve Mimarlik Fakultesi

Bilgisayar Muhendisligi Bolumu

GOMULU SISTEMLER

AMBULANS SIMULASYON SISTEMI

ESP32-S3 Tabanlı Çok Modlu Ambulans

Işık ve Siren Similatörü

Oğrenci Adı: Yakup Eroğlu

Oğrenci Numarası: 221401045

Ders: Gomu lu Sistemler

Platform: ESP32-S3 | PlatformIO

Tarih: Haziran 2026

ICERIK TABLOSU

1 Projeye Genel Bakis

2 Kullanilan Donanim

2.1 ESP32-S3-DevKitC-1

2.2 Joystick Modulu

2.3 Hoparlör Modulu

2.4 NeoPixel LED Ring

3 Devre Baglantilari

4 Yazilim Mimarisi

5 Calisma Modlari

5.1 Mod 0 -- Kapali

5.2 Mod 1 -- Klasik Ambulans

5.3 Mod 2 -- Tam Renk Dongusu

5.4 Mod 3 -- Yavas Flas

6 Fonksiyon Referansi

7 Non-Blocking Zamanlama

8 PWM Ses Uretimi

9 NeoPixel LED Kontrolu

10 Buton Debounce

11 Gelistirme Ortami

12 Seri Monitor Ciktilari

13 Kisitlamalar

14 Gelecek Gelistirme Fikirleri

1. PROJEYE GENEL BAKIS

Bu proje, ESP32-S3 mikrodnetleyicisi kullanilarak PlatformIO platformu uzerinde Arduino cercevesiyle gelistirilmis cok modlu bir ambulans isik ve siren simulasyon sistemidir. Sistem, joystick butonuyla kontrol edilen dort farkli calisma modunda donerli olarak calismakta; gercekci ambulans tipi LED animasyonlari ve siren sesleri uretmektedir.

Hedefler

- ESP32-S3 mikrodnetleyici kullanarak gercek zamanli gomulu sistem tasarlamak ve uygulamak
- delay() kullanmadan millis() tabanli non-blocking yazilim mimarisini gosterme
- WS2812B adreslenebilir RGB LED'leri birden fazla animasyon kalibinda kontrol etmek
- PWM tabanli ses tonlariyla ambulans siren seslerini simule etmek
- 50ms yazilimsal debounce algoritmasi ile guvenilir buton okuma saglamak
- Gomulu C++ en iyi uygulamalarini izleyen moduler ve okunabilir kod yazmak

Sistem Akisi

```

Baslat
|
v
setup() -> Donanim Baslat -> Baslangic Animasyonu -> MOD 0 (Kapali)
|
v
loop() [surekli calisir]
|-- Buton basildi mi? --> enterMode(sonrakiMod) [0->1->2->3->0...]
|-- MOD 1 --> updateModel(now) [bolunmus LED + wee-woo siren]
|-- MOD 2 --> updateMode2(now) [tam renk LED + cift bip]
+-- MOD 3 --> updateMode3(now) [3 sn bloklar + agir siren]

```

2. KULLANILAN DONANIM

2.1 ESP32-S3-DevKitC-1

Projenin ana isleme birimi olan ESP32-S3-DevKitC-1, Espressif ESP32-S3 SoC uzerine insaa edilmis guclu bir gelistirme kartidir. 240 MHz hizinda cift cekirdekli Xtensa LX7 islemcisi ve GPIO48'de dahili WS2812B RGB LED icermektedir.

Ozellik	Deger
Islemci	Xtensa LX7 cift cekirdek, 240 MHz
RAM	320 KB SRAM
Flash	8 MB (Quad SPI)
Dahili RGB LED	GPIO48 (WS2812B)
ADC	12-bit, 20 kanal
PWM (LEDC)	8 kanal, 14-bit'e kadar
USB	USB-OTG (programlama ve seri monitor)

2.2 Deneyap Joystick Modulu

iki eksenli analog joystick ve entegre basan buton iceren moduldur. Bu projede yalnızca SW (basan buton) pini kullanılmaktadır. X ve Y analog cikislari fiziksel olarak baglidir ancak gelecek gelistirmeler icin rezervedir.

Pin	GPIO	Fonksiyon
SW (Buton)	GPIO3	Mod degistirme -- INPUT_PULLUP
X (Analog)	GPIO1	Kullanilmiyor (rezerve)
Y (Analog)	GPIO2	Kullanilmiyor (rezerve)

2.3 Deneyap Hoparlor Modulu

Uzerinde hoparlor ve amplifikator entegre kucuk moduldur. ESP32-S3 LEDC birimi tarafından PWM sinyaliyle suruulmektedir. SD (shutdown) pini MOD 0'da LOW'a cekilir; hoparlor tamamen kapatilarak bosta gurultu ve guc tuketimi engellenir.

Pin	GPIO	Fonksiyon
SD	GPIO4	OUTPUT -- HIGH=acik, LOW=kapali
IN+	GPIO5	PWM Kanal 0 ses sinyali
GND	GND	Ortak toprak

2.4 WS2812B NeoPixel LED Ring (8 LED)

Dairesel duzenlemede 8 bagimsiz adreslenebilir RGB LED. LED basina 24-bit renk (R, G, B her biri 8-bit). VCC mutlaka 5V olmalidir -- 3.3V ile renk bozulmasi ve yetersiz parlaklik yasanir.

Ozellik	Deger
LED Sayisi	8
Protokol	WS2812B tek-hat, 800 KHz
Renk Derinligi	24-bit RGB (kanal basina 8-bit)
Calisma Gerilimi	5V (sinyal 3.3V toleransli)
Maks. Akim	~480 mA (tum LED'ler tam beyaz)
Kutuphane	Adafruit NeoPixel 1.15.5

3. DEVRE BAGLANTILARI

Modul	Modul Pini	ESP32-S3 GPIO	Mod	Aciklama
Joystick	3V3	3V3	Guc	Besleme gerilimi
Joystick	GND	GND	Toprak	Ortak toprak
Joystick	SW	GPIO3	INPUT_PULLUP	Basan buton (mod degistirme)
Joystick	X	GPIO1	ADC (kullanilmiyor)	X ekseni analog
Joystick	Y	GPIO2	ADC (kullanilmiyor)	Y ekseni analog
Hoparlör	GND	GND	Toprak	Ortak toprak
Hoparlör	SD	GPIO4	OUTPUT	Etkin/kapali kontrolu
Hoparlör	IN+	GPIO5	PWM CH0	Ses kare dalga sinyali
LED Ring	GND	GND	Toprak	Ortak toprak
LED Ring	VCC	5V	Guc	USB'den 5V
LED Ring	DIN	GPIO6	NeoPixel	WS2812B veri sinyali
Dahili RGB	--	GPIO48	Cikis	Kart üzerindeki RGB LED

4. YAZILIM MIMARISI

Pin Tanimlari

```
#define JOY_SW_PIN      3    // Joystick basan buton
#define SPEAKER_SD_PIN  4    // Hoparlör enable/shutdown
#define SPEAKER_PWM_PIN 5    // Hoparlör PWM sinyali
#define LED_PIN         6    // WS2812B NeoPixel veri
#define LED_COUNT       8    // LED sayisi
#define RGB_LED_PIN     48    // Dahili RGB LED
#define PWM_CHANNEL     0    // LEDC kanali
#define PWM_RESOLUTION  8    // 8-bit cozunurluk
```

Temel Global Degiskenler

Degisken	Tip	Rol
currentMode	int	Aktif mod numarasi (0-3)
lastBtnState / debouncedBtn	bool	Debounce durum takibi
lastDebounce	unsigned long	Debounce zamanlayicisi (ms)
lastLedUpdate	unsigned long	LED animasyon zamanlayicisi (ms)
flashState	bool	LED renk faz bayragi
lastSoundUpdate	unsigned long	Ses guncelleme zamanlayicisi (ms)
currentFreq / sirenDir	int	Siren sweep durumu
bipPhase / bipTimer	int / ulong	Mod 2 çift bip durum makinesi
blockStart / blockRed	ulong / bool	Mod 3 3 saniyelik blok durumu

5. CALISMA MODLARI

Sistem, butona her basista bir sonraki moda gecir: MOD 0 -> 1 -> 2 -> 3 -> 0 ...

Mod 0 -- Kapali (Bekleme)

Guc verildikten sonraki varsayilan durumdur. Tum cikislar tamamen pasiftir. Hoparlör SD pini LOW'dur (amplifikator kapali). Yalnizca buton her dongu adiminda okunur.

Mod 1 -- Klasik Ambulans

Klasik Avrupa ambulansini simule eder. LED 0-3 ve LED 4-7, her 120 ms'de bir mavi ile kirmizi arasinda alternatif olarak yanar. Siren 700 Hz'den 1200 Hz'e ve geri iner (~1.75 Hz tam dongu).

Ozellik	Detay
LED Kalibi	Sol 4: Mavi, Sag 4: Kirmizi -- 120 ms'de bir taraflar degisir
Siren Araligi	700 Hz'den 1200 Hz'e
Sweep Adimi	Her 8 ms'de 14 Hz
Tam Wee-Woo Dongusu	~572 ms

Mod 2 -- Tam Renk Dongusu

8 LED'in tamamı birlikte yanar: 200 ms'de bir tam mavi, tam kirmizi. Ses, 780 ms'lik dongu suresinde iki tonlu çift bip kalibi calar.

Bip Fazı	Frekans	Sure
Faz 0 (Ton 1)	900 Hz	150 ms
Faz 1 (Sessiz)	--	80 ms
Faz 2 (Ton 2)	600 Hz	150 ms
Faz 3 (Sessiz)	--	400 ms
Toplam Dongu	--	780 ms

Mod 3 -- Yavas Flas

Tum LED'ler 3 saniye boyunca kirmizi yanar, ardindan 3 saniye boyunca maviye gececek. Siren, LED renk degisimiyle senkronize olarak ~3 saniyede bir tam dongu yaparak 500 Hz'den 1000 Hz'e kadar yavasca kayar.

Ozellik	Detay
LED Kalibi	Tum LED'ler 3 sn KIRMIZI, sonra 3 sn MAVI
Siren Araligi	500 Hz'den 1000 Hz'e
Sweep Adimi	Her 18 ms'de 6 Hz
Tam Sweep Dongusu	~3000 ms (LED blok suresiyle eslesir)

6. FONKSIYON REFERANSI

setRGB(r,g,b)

GPIO48 dahili RGB LED'i neopixelWrite() ile kontrol eder. Ek kutuphane gerekmez.

speakerOn(freq)

ledcWriteTone() ile LEDC CH0 frekansini ayarlar. Minimum 20 Hz guvenlik siniri icerir.

speakerOff()

LEDC duty'yi 0'a dusurur. SD pin durumunu degistirmez.

allLed(r,g,b)

8 NeoPixel'in tamamini doldurur ve ring.show() ile kopyalar.

ledOff()

Ring'i temizler ve dahili RGB'yi sondurur. MOD 0 girisinde cagirilir.

enterMode(mode)

Tum durum degiskenlerini sifirlar, SD pinini kontrol eder, yeni modu temiz baslatir.

updateModel1(now)

MOD 1 mantigi: LED her 120ms, sweep her 8ms -- tamamen non-blocking.

updateMode2(now)

MOD 2 mantigi: LED her 200ms, 4 fazli bip durum makinesi.

updateMode3(now)

MOD 3 mantigi: LED bloku her 3000ms, statik degisim algilamasiyla.

readButtonPressed()

50ms debounce; yalnızca gerçek basista bir kez true doner.

setup()

Bir defaya mahsus baslangic: seri, GPIO, LEDC, NeoPixel, animasyon.

loop()

Buton kontrol, mod guncelleme fonksiyonu cagrisi, delay(1) CPU molasi.

7. NON-BLOCKING ZAMANLAMA TASARIMI

delay() fonksiyonu tum program yurutmesini durdurur; LED, ses ve buton ayni anda yönetilemez. Tum zamanlama millis() tabanlı non-blocking kontrollerle yapılır:

```
// YANLIS -- 120ms boyunca buton ve ses okunamaz:
ring.fill(mavi); ring.show(); delay(120);

// DOGRU -- <1ms'de geri doner, her sey calismaya devam eder:
if (now - lastLedUpdate >= 120) {
    lastLedUpdate = now;
    // LED guncelle
}
```

Gorev	Aralik	Degisken
Mod 1 LED flas	120 ms	lastLedUpdate
Mod 1 siren sweep	8 ms	lastSoundUpdate
Mod 2 LED flas	200 ms	lastLedUpdate

Mod 2 bip fazi	80-400 ms	bipTimer
Mod 3 LED blok	3000 ms	blockStart
Mod 3 siren sweep	18 ms	lastSoundUpdate
Buton debounce	50 ms	lastDebounce

8. PWM SES URETIMI

ESP32-S3 LEDC cevre birimi, harici DAC veya I2S modulu gerekmeksizin kare dalga ton uretir. Kare dalganin harmonik icerigi siren sesine aslinda gerce bir sertlik katar. Kurulan platform surumu (espressif32 @ 7.0.1) eski LEDC API'si gerektirdiginden ledcSetup + ledcAttachPin kombinasyonu kullanilmaktadir.

```
// Kurulum (setup() icinde bir kez):
ledcSetup(PWM_CHANNEL, 2000, PWM_RESOLUTION);
ledcAttachPin(SPEAKER_PWM_PIN, PWM_CHANNEL);

// Ton ac/kapat:
ledcWriteTone(PWM_CHANNEL, 880); // 880 Hz kare dalga
ledcWrite(PWM_CHANNEL, 0); // sessiz (duty = 0)

// Sweep algoritmasi (her N ms'de loop icinde calisir):
currentFreq += sirenDir * ADIM;
if (currentFreq >= MAX) { currentFreq = MAX; sirenDir = -1; }
if (currentFreq <= MIN) { currentFreq = MIN; sirenDir = +1; }
speakerOn(currentFreq);
```

Mod	Min Hz	Max Hz	Adim	Aralik	Karakter
Mod 1	700	1200	14 Hz	8 ms	Hizli wee-woo (~1.75 Hz)
Mod 3	500	1000	6 Hz	18 ms	Agir yavas siren (~0.33 Hz)

9. NEOPIXEL LED KONTROLU

WS2812B LED'ler tek hat 800 KHz seri protokol kullanir. Her LED 24 bit alir (R, G, B her biri 8-bit) ve geri kalan veriyi asagi gonderir. ring.show() her LED degisikliginden sonra mutlaka cagirilmalidir -- degisiklikler o ana kadar buffer'da kalir.

```
Adafruit_NeoPixel ring(8, LED_PIN, NEO_GRB + NEO_KHZ800);
ring.begin();
ring.setBrightness(130); // ~%51 parlaklik
ring.setPixelColor(i, ring.Color(r,g,b)); // tek LED
ring.fill(ring.Color(255, 0, 0)); // tum LED'ler
ring.clear(); // tum LED kapal
ring.show(); // donanima isle (ZORUNLU)
```

10. BUTON DEBOUNCE

Mekanik butonlar basilip birakildiginda 1-50 ms icinde hizli ve gurultulu gecisler (bounce) uretir. Debounce olmadan tek bir basisi cok sayida mod degisikligine yol acar. 50 ms millis tabanlı algoritma bu problemi cozer:

```
bool readButtonPressed() {
    bool reading = digitalRead(JOY_SW_PIN);
    if (reading != lastBtnState) { lastDebounce = millis(); lastBtnState = reading; }
    if ((millis() - lastDebounce) > DEBOUNCE_MS) {
        if (reading != debouncedBtn) {
            debouncedBtn = reading;
            if (debouncedBtn == LOW) return true; // Gercek basisi!
        }
    }
    return false;
}
```

11. GELISTIRME ORTAMI

Yazilim	Surum	Amac
Visual Studio Code	Herhangi	Kod editoru
PlatformIO IDE	Herhangi	Derleme ve yukleme sistemi
Espressif32 Platform	7.0.1	ESP32 arac zinciri
Arduino Framework	3.20017.241212	Arduino uyumluluk katmani
Adafruit NeoPixel	1.15.5	WS2812B LED surucusu

```
[env:esp32-s3-devkitc-1]
platform = espressif32
board = esp32-s3-devkitc-1
framework = arduino
lib_deps = adafruit/Adafruit NeoPixel @ ^1.12.3
monitor_speed = 115200
upload_speed = 921600
```

12. SERI MONITOR CIKILARI

```
Ambulans v2 -- 3 Mod + Kapali
  Buton: MOD 0->1->2->3->0->...
  MOD 0: Kapali
  MOD 1: Klasik ambulans
  MOD 2: Tam mavi-kirmizi + cift bip
  MOD 3: 3sn bloklar + agir siren
  Hazir! MOD 0 (Kapali). Butona bas.

  MOD 1 aktif
  MOD 2 aktif
  MOD 3 aktif
  -> KIRMIZI
  -> MAVI
  MOD 0 -- Kapali
```

13. BILINEN KISITLAMALAR

- Eski LEDC API: espressif32 3.x'e yükseltme ledcAttachChannel() gecisini gerektirir.
- Kare dalga sesi harmonik bozulma içerir; temiz ses için I2S + sinus LUT gerekir.
- Joystick X/Y okunmuyor -- bağlı ancak gelecek hız/parlaklık kontrolü için rezerve.
- Kalıcı mod belleği yok; kart her açıldığında MOD 0'dan başlar.

14. GELECEK GELISTIRME FIKIRLERI

Kisa Vadeli

- Joystick X/Y okuma ile siren hızı ve LED parlaklık kontrolü
- Mod 4: Gökkuşu HSV spektrum animasyonu
- NVS kalıcı mod belleği

Orta Vadeli

- OLED ekran (SSD1306) ile mod ve frekans gosterimi
- Bluetooth BLE ile akilli telefon mod kontrolu
- I2S DAC (MAX98357A) ile daha temiz siren

Uzun Vadeli

- MAX30102 kalp atisi entegrasyonu
- Wi-Fi web arayuzu (AsyncWebServer)
- Pil + derin uyku ile tasinalir kullanim

HIZLI REFERANS KARTI

Ambulans Simulasyon Sistemi -- ESP32-S3

PIN ATAMALARI

BUTON (SW)	GPIO3	INPUT_PULLUP -- Mod degistirme
HOPARLOR SD	GPIO4	OUTPUT -- Enable/shutdown
HOPARLOR IN+	GPIO5	PWM Kanal 0
LED VERI	GPIO6	NeoPixel DIN
DAHILI RGB	GPIO48	neopixelWrite()

CALISMA MODLARI

MOD 0	KAPALI	Tum cikislar pasif -- bekleme
MOD 1	KLASIK	Bolunmus mavi/kirmizi flas + wee-woo
MOD 2	TAM RENK	Tum mavi <-> kirmizi + cift bip
MOD 3	YAVAS FLAS	3sn renk bloklari + agir yavas siren